



Course Title	: FRONT END DEVELOPMENT FRAMEWORKS AND UI ENGINEERING
Course Code	: 25CS1201E
L-T-P-S Structure	: 0-0-4-4
Pre-requisite	:
Credits	: 3
Course Coordinator	: Balajee R M
Team of Instructors	:
Teaching Associates	:
Syllabus :	CO1: Foundations of Front-End Engineering & Framework Design: The problem frameworks solve; imperative vs declarative UIs; engineering constraints of front-end systems; DOM limitations; Virtual DOM as an engineering abstraction; unidirectional data flow; component-driven thinking; rendering pipeline; reconciliation (engineering logic, not React.exe); how frameworks abstract complexity; architectural concepts enabling React-like frameworks (composition, immutability, reactive state). CO2: JavaScript & TypeScript Engineering for Frameworks CO2 ES6+ essentials for framework engineering; closures as memory/state constructs; functional programming in UI; immutability and pure functions; module patterns; async/await and promise chains for UI updates; error boundaries at JS level; TS types/interfaces/generics to enforce UI correctness; architecture of a front-end codebase (folder organization, modules, services). CO3: React Component Model (Engineering View, Not Syntax View) Component as deterministic UI function; props as configuration contract; state as dynamic runtime data; side-effects as controlled event loops; React hooks as abstractions over lifecycle; reconciliation: how React decides to re-render; controlled vs uncontrolled UI engineering; component composition patterns; designing reusable components; styling approaches: CSS modules, styled frameworks, Tailwind (engineering reasoning for each). CO4. State Architecture, Async Data Engineering & API Integration: Lifting state; state co-location; derived state; global state requirements; React Context engineering rationale; API service layers; async flow control; race conditions; stale state problems; data caching strategies; skeleton UIs; architectural patterns: container-presenter, smart-dumb components; error boundaries in React; handling complex UI events and async sequencing. CO5. Routing, Forms, Accessibility & Performance Engineering: SPA routing fundamentals; dynamic & nested routing; rendering boundaries; protected routes; form engineering: validation pipelines, controlled forms, error surfaces; accessibility engineering: ARIA, keyboard flow, color contrast; performance tuning: virtualization, memoization, keying, batching renders, lazy loading; architectural tradeoffs in performance engineering. CO6. Build Systems, Testing, CI/CD & Deployment Engineering: Why bundlers exist; Vite/Webpack engineering flow; bundling, tree-shaking, minification, asset optimization; environment configs; error reporting; linting/formatting; React testing (unit, integration) with Jest & RTL; CI/CD pipeline design; cloud deployment (Netlify/Vercel); monitoring production; performance evaluation through Lighthouse; engineering practices for maintainability and scalability.
Text Books :	1. The Road to React – Robin Wieruch, Modern engineering-first React introduction 2. JavaScript: The Definitive Guide – David Flanagan, Deep JS concepts for framework engineers

Reference Books :	1. TypeScript Quickly – Yakov Fain, TS engineering foundations 2. Learning JavaScript Design Patterns – Addy Osmani, Reusable component design & architecture 3. Designing Interfaces – Jenifer Tidwell, UI engineering & interaction patterns
MOOCS :	1. Front End Development Libraries V8 - https://www.freecodecamp.org/learn/front-end-development-libraries/ 2. Codedamn – Learn React 18 - https://codedamn.com/learn/reactjs?utm_source=chatgpt.com
Course Rationale :	Modern front-end development requires strong engineering and architectural thinking beyond framework syntax. This course adopts an architecture-first approach to explain why front-end frameworks exist and how they address complexity, performance, and scalability challenges. It emphasizes declarative UI design, component-based architectures, state management, and asynchronous data handling. Learners gain the ability to reason about rendering, performance, and maintainability in production-grade front-end systems.
Course Objectives :	The objective of this course is to enable learners to understand front-end frameworks from an engineering perspective and design scalable, maintainable UI architectures. Students will apply advanced JavaScript and TypeScript concepts to build correct and predictable UI systems. The course develops skills in component design, state architecture, routing, performance optimization, accessibility, and modern build and deployment workflows. Learners will be prepared to engineer and evaluate real-world front-end applications.
Global Certifications :	1. Full stack open certificate - https://fullstackopen.com/en/
Industry Specific Tools :	1. VS Code - https://code.visualstudio.com/download 2. NodeJS - https://nodejs.org/en/download
Sector Skill Council :	National Skill Development Corporation – The Front-End Developer job role under IT-ITeS SSC, with QP Code SSC/Q1401, is defined as NSQF Level 6.

COURSE OUTCOMES (COs):

CO NO	Course Outcome (CO)	PO/PSO	Blooms Taxonomy Level (BTL)
CO1	Analyze modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.	PO5,PSO1,PO3	4
CO2	Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.	PO3,PO5,PSO1	3
CO3	Build React applications using component-driven development, state and props architecture, side-effect management, reconciliation fundamentals, and controlled UI patterns.	PO3,PO5,PSO1	3
CO4	Apply front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns like lifting state, composition, and container–presenter design.	PO3,PSO1,PO5	3
CO5	Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.	PO3,PO5,PSO1	4
CO6	Assess front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines,	PO3,PO5,PO9,PSO1	5

CO NO	Course Outcome (CO)	PO/PSO	Blooms Taxonomy Level (BTL)
	CI/CD, and cloud deployment workflows.		

COURSE OUTCOME INDICATORS (COIs)::

Outcome No.	Highest BTL	COI-1	COI-2	COI-3	COI-4	COI-5
CO1	4	Btl-1 Remember modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.	Btl-2 Understand modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.	Btl-3 Apply modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.	Btl-4 Analyze modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.	
CO2	3	Btl-1 Remember JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.	Btl-2 Understand JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.	Btl-3 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.		
CO3	3	Btl-1 Remember React applications using component-driven development, state and props architecture, side-effect management,	Btl-2 Understand React applications using component-driven development, state and props architecture, side-effect management,	Btl-3 Build React applications using component-driven development, state and props architecture, side-effect management, reconciliation		

Outcome No.	Highest BTL	COI-1	COI-2	COI-3	COI-4	COI-5
		reconciliation fundamentals, and controlled UI patterns.	reconciliation fundamentals, and controlled UI patterns.	fundamentals, and controlled UI patterns.		
CO4	3	Btl-1 Remember front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns like lifting state, composition, and container–presenter design.	Btl-2 Understand front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns like lifting state, composition, and container–presenter design.	Btl-3 Apply front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns like lifting state, composition, and container–presenter design.		
CO5	4	Btl-1 Remember routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.	Btl-2 Understand routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.	Btl-3 Apply routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.	Btl-4 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.	
CO6	5	Btl-1 Remember front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.	Btl-2 Understand front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.	Btl-3 Apply front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.	Btl-4 Analyze front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.	Btl-5 Assess front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.

PROGRAM OUTCOMES & PROGRAM SPECIFIC OUTCOMES (POs/PSOs)

Po No.	Program Outcome
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO9	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PSO1	Ability to design, develop, and test software solutions using computational engineering principles.

Lecture Course DELIVERY Plan:

NO Delivery Plan Exists

Lecture Session wise Teaching – Learning Plan

No Session Plans Exists

Tutorial Course DELIVERY Plan:

NO Delivery Plan Exists

Tutorial Session wise Teaching – Learning Plan

No Session Plans Exists

Practical Course DELIVERY Plan:

Tutorial Session no	Topics	CO-Mapping
1	imperative vs declarative UIs; engineering constraints of front-end systems; unidirectional data flow; component-driven thinking; rendering pipeline; reconciliation (engineering logic, not React.exe);	CO1
2	Understanding React Framework Abstractions through Component-Based UI Rendering Scenario: You are designing a Profile Dashboard for a web application where a user's details (name, role, and status) are displayed and updated dynamically. The objective is to understand why front-end frameworks are required, how they abstract DOM complexity, and how declarative, component-driven design improves engineering quality compared to imperative UI manipulation. This experiment demonstrates Virtual DOM concepts, unidirectional data flow, and reconciliation logic through controlled state updates. Task: • Create a React application using functional components. • Design reusable components such as ProfileCard and StatusPanel. • Manage UI data using React state and render updates declaratively. • Pass data from parent to child components to enforce unidirectional data flow. • Update the UI through state changes instead of direct DOM access. • Observe React's rendering and reconciliation behavior conceptually. Skills to be Included: ✓ Framework abstraction and DOM limitations ✓ Virtual DOM and reconciliation (conceptual) ✓ Imperative vs declarative UI patterns ✓	CO1

Tutorial Session no	Topics	CO-Mapping
	Component-driven thinking ✓ Unidirectional data flow ✓ Front-end engineering constraints	
3	ES6+ essentials for framework engineering	CO2
4	Creating a Modular React Application with Reusable Components Scenario: You are developing a User Information Dashboard for a front-end application. The objective is to understand how a React application is created and organized, how front-end code architecture improves maintainability, and how reusable components and ES6 module patterns are used in real-world framework engineering. The experiment focuses on project structure, component reuse, and modern JavaScript (ES6+) concepts, rather than advanced UI design. Task: • Create a React application using a standard project setup. • Organize the project using a clean front-end architecture (components and modules). • Use ES6 module patterns (import and export) to connect components. • Apply ES6+ features such as arrow functions, destructuring, and default exports. • Render components in a parent container to form a complete UI. Skills to be Included: ✓ Creating a React application ✓ Front-end codebase architecture (folders, modules) ✓ Designing reusable React components ✓ ES6 module patterns (import / export) ✓ ES6+ JavaScript essentials for framework engineering	CO2
5	error boundaries at JS level; TS types/interfaces/generics to enforce UI correctness;	CO2
6	Asynchronous Product Fetching and Component Architecture Using Public Products API Scenario: You are developing a Product Listing Page that fetches product data from a public products API (https://dummyjson.com/products). While the product data is loading, a skeleton UI must be displayed to improve user experience. This experiment focuses on handling asynchronous UI updates using async/await and promise chains, applying closures as memory/state constructs, and designing deterministic React components using proper architectural patterns such as container–presenter (smart–dumb) components. Task: • Fetch product data from https://dummyjson.com/products using async/await and promise chaining. • Use a closure to track the number of API fetch attempts or the last fetch timestamp. • Implement a container (smart) component to handle data fetching and state management. • Implement presentational (dumb) components to display product details using props only. • Display a skeleton UI while products are loading. • Use React hooks (useState, useEffect) and observe re-rendering when state updates occur. Skills to be Included: ✓ Async/await and promise chains for UI updates ✓ Closures as memory/state constructs ✓ Component as a deterministic UI function (props, state) ✓ React hooks and re-rendering behavior ✓ Skeleton UI implementation ✓ Container–presenter (smart–dumb) architectural pattern	CO2
7	Controlled vs uncontrolled UI engineering; Component composition patterns	CO3
8	Styling Strategies and Component Composition in React Scenario: You are enhancing a previously created React page (such as a product or profile page) to improve its visual design and maintainability. The goal is to explore different styling approaches used in React applications and understand engineering reasons behind choosing frameworks like Tailwind CSS to improve design speed. This experiment also demonstrates controlled vs	CO3

Tutorial Session no	Topics	CO-Mapping
	uncontrolled UI engineering using a single text input, and applies component composition patterns to build a clean and reusable UI structure. Task: • Style the existing page using CSS Modules to achieve scoped and maintainable styles. • Apply Tailwind CSS to improve the pace of UI design and consistency. • Justify the engineering reasoning behind using Tailwind (speed, consistency, scalability). • Implement one controlled text input using React state (useState). • Implement one uncontrolled text input using useRef (for comparison). • Compose the UI using component composition patterns (parent–child, wrapper components). Skills to be Included: ✓ Styling approaches: CSS Modules and Tailwind CSS ✓ Engineering reasoning for choosing Tailwind ✓ Page design improvement using modern styling strategies ✓ Controlled vs uncontrolled UI engineering (single text input) ✓ Component composition patterns	
9	React hooks and rerendering Lifting state; state co-location; derived state; global state requirements;	CO4
10	Designing Sign-In and Sign-Up Pages with State Management in React Scenario: You are developing Sign-In and Sign-Up pages for a web application. The objective is to understand how React hooks trigger re-rendering, how state should be placed correctly (state co-location), and when state needs to be lifted to a common parent component. This experiment also explores derived state (computed UI values) and analyzes when global state management may be required in real-world applications. Task: • Design Sign-In and Sign-Up UI components using React functional components. • Use React hooks (useState, useEffect) to manage input values and UI state. • Observe component re-rendering when state changes. • Apply state co-location by keeping state close to where it is used. • Lift state to a parent component to share authentication status between Sign-In and Sign-Up. • Implement derived state (e.g., enabling/disabling submit button based on input validity). • Discuss scenarios where global state would be required for authentication. Skills to be Included: ✓ Designing Sign-In and Sign-Up pages ✓ React hooks and re-rendering behavior ✓ State lifting and state co-location ✓ Derived state implementation ✓ Understanding global state requirements	CO4
11	API service layers; async flow control; race conditions; stale state problems; data caching strategies;	CO4
12	Enhancing Product Page with Advanced Features API Data Source (Product Data): URL: https://dummyjson.com/products Continuing from the Week 3 Product Listing Page project. Scenario: This experiment builds upon the Week 3 Product Page project. You will enhance the existing product listing page with additional features like improved caching, handling stale state, race conditions, and preparing the architecture for future routing and nested routing. It continues to focus on async UI updates, closures, component composition, and smart–dumb architectural patterns, while adding data caching and state safety improvements. Task: • Fetch product data from https://dummyjson.com/products using async/await and promise chains (as before). • Use a closure to track API fetch attempts or last fetch timestamp. • Implement a container (smart) component to manage data fetching, state updates, and caching. • Implement presentational (dumb) components to display product details via props. • Display a skeleton UI while products are loading. • Handle race conditions when multiple API calls are triggered	CO4

Tutorial Session no	Topics	CO-Mapping
	simultaneously. • Prevent stale state problems when delayed API responses update the UI. • Implement simple in-memory caching to minimize repeated API calls. • Organize components and state to support future routing and nested routing. Skills to be Included: ✓ Async/await and promise chains for UI updates ✓ Closures as memory/state constructs ✓ Component as a deterministic UI function (props, state) ✓ React hooks and re-rendering behavior ✓ Skeleton UI implementation ✓ Container–presenter (smart–dumb) architectural pattern ✓ Handling race conditions and stale state ✓ Basic in-memory data caching ✓ Preparation for routing and nested routing	
13	form engineering: validation pipelines, controlled forms, error surfaces;	CO5
14	Implementing SPA Routing and Form Engineering in React Scenario: You are developing a Single Page Application (SPA) that includes multiple pages: Home, Products, and Contact. Users should navigate between pages without full page reloads, and some pages (e.g., Contact) should be protected based on login status. The application also contains a user feedback form, where inputs must be validated using controlled components, and validation errors displayed on the UI. The goal is to understand dynamic and nested routing, rendering boundaries, protected routes, and form engineering best practices. Task: • Set up SPA routing using react-router-dom. • Implement dynamic routes for product details (/products/:id). • Implement nested routes (e.g., /products/:id/reviews). • Create protected routes for pages that require login simulation. • Build a controlled form with fields such as name, email, and message. • Apply form validation pipelines and display error messages / error surfaces. • Demonstrate rendering boundaries to prevent unnecessary re-renders of unrelated components. Skills to be Included: ✓ SPA routing fundamentals ✓ Dynamic and nested routing ✓ Rendering boundaries for performance optimization ✓ Protected routes based on login state ✓ Controlled forms in React ✓ Form validation pipelines and error surfaces	CO5
15	Context API, Browser Storages	CO5
16	Managing Global State Using React Context API Scenario: You are developing a multi-component React application where certain state (e.g., user authentication status, theme preference, or shopping cart) must be shared across multiple components. Instead of prop drilling, the application will use React Context API to manage global state efficiently. The experiment focuses on creating contexts, providing and consuming values, updating global state, and understanding context re-rendering behavior. Task: • Create a Context for shared state (e.g., user login status or theme). • Wrap the application with the Context Provider to make state available to child components. • Consume the context in nested components using useContext. • Update global state from a child component and observe changes reflected across all consuming components. • Demonstrate re-render behavior when context values change. • Optionally, create multiple contexts (e.g., one for theme, one for cart) and handle independent updates. Skills to be Included: ✓ Creating and providing React Context ✓ Consuming context values using useContext ✓ Updating global state from nested components ✓ Understanding re-rendering implications of context changes ✓ Managing multiple contexts independently	CO5
17	React testing (unit, integration) with Jest & RTL;	CO6

Tutorial Session no	Topics	CO-Mapping
18	Unit and Integration Testing of React Components with Jest and React Testing Library Scenario: You are developing a React application with multiple components such as a Product Card, Product List, and Cart Summary. To ensure reliability, you need to write unit and integration tests for these components using Jest and React Testing Library (RTL). The experiment focuses on testing component behavior, state updates, props, events, and interactions between components. Task: • Set up Jest and React Testing Library in the React project. • Write unit tests for individual components (e.g., ProductCard: rendering props, click events). • Write integration tests for multiple components working together (e.g., ProductList + ProductCard). • Test user interactions such as clicking buttons, typing in inputs, and form submission. • Assert that state changes and UI updates happen correctly. • Run tests and analyze test coverage reports. Skills to be Included: ✓ Unit testing React components with Jest ✓ Integration testing with React Testing Library ✓ Testing props, events, and state updates ✓ Testing user interactions and controlled components ✓ Writing assertions for DOM updates ✓ Understanding test coverage and reliability	CO5
19	Redux Architecture Need: More Security	CO6
20	Packaging and Deployment of SPA with Forms Prerequisite: SPA Routing and Form Engineering in React Students should have a working SPA with routing, nested routes, protected routes, and validated controlled forms. Scenario: You have a fully functional React SPA from Exp. 7. Now, the application needs to be packaged and deployed for production. This involves bundling, optimizing assets, configuring environments, and deploying to cloud platforms. The experiment focuses on: • Understanding why bundlers (Vite/Webpack) exist • Optimizing assets and code for production • Configuring environment variables and linting • Deploying the SPA via Netlify or Vercel • Evaluating performance and accessibility using Lighthouse • Applying engineering best practices for maintainable and scalable deployments Task: • Use the SPA from previous exp. as the base project. • Build the project using Vite or Webpack, observing the bundling and optimization process. • Enable tree-shaking, minification, and asset optimization. • Configure environment variables for development and production modes. • Run linting and formatting to maintain code quality. • Deploy the SPA to a cloud platform such as Netlify or Vercel. • Analyze performance, accessibility, and best practices using Lighthouse. • Monitor for errors in production and plan for maintainable updates. Skills to be Included: ✓ Using SPA from Exp. 7 as a base for production ✓ Bundling with Vite/Webpack ✓ Tree-shaking, minification, and asset optimization ✓ Environment configuration for different stages ✓ Linting and formatting ✓ Cloud deployment (Netlify/Vercel) ✓ Performance evaluation using Lighthouse ✓ Engineering practices for maintainable and scalable applications	CO6

Practical Session wise Teaching – Learning Plan

SESSION NUMBER : 1

Session Outcome: 1 Analyze modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 2

Session Outcome: 1 Analyze modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementation of Experiment	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 3

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 4

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 5

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 6

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 7

Session Outcome: 3 Build React applications using component-driven development, state and props architecture, side-effect management, reconciliation fundamentals, and controlled UI patterns.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 8

Session Outcome: 3 Build React applications using component-driven development, state and props architecture, side-effect management, reconciliation fundamentals, and controlled UI patterns.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 9

Session Outcome: 4 Apply front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 10

Session Outcome: 4 Apply front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 11

Session Outcome: 4 Apply front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 12

Session Outcome: 4 Apply front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 13

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 14

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 15

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 16

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 17

Session Outcome: 6 Assess front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	5	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 18

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 19

Session Outcome: 6 Assess front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 20

Session Outcome: 6 Assess front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Discussing the Topic	2	PPT	--- NOT APPLICABLE ---
50	Implementing the Concept	3	LTC	--- NOT APPLICABLE ---

Skilling Course DELIVERY Plan:

Skilling session no	Topics/Experiments	CO-Mapping
1	The problem frameworks solve; how frameworks abstract complexity;	CO1
2	DOM limitations; Virtual DOM as an engineering abstraction;	CO1
3	Creating React Application architecture of a front-end codebase (folder organization, modules, services). Need due to: Solution to the Project Statement using App Creation	CO1
4	Designing reusable components module patterns (import and export) Need due to: Multi Page Design	CO2
5	async/await and promise chains for UI Updates closures as memory/state constructs;	CO2
6	Implementing promise and memory closure in project Component as deterministic UI function; props, state React hooks and rerendering skeleton UIs; architectural patterns: container-presenter, smart-dumb components; Need	CO2

Skilling session no	Topics/Experiments	CO-Mapping
	due to: 3rd party data fetching and dynamic update of page data Performance Improvement and Gaining User Confidence	
7	styling approaches: CSS modules, styled frameworks Need due to: Designing of Created Page	CO2
8	Tailwind (engineering reasoning for each). Need due to: Improve pace of Design in Created Page	CO2
9	Designing Signin and Signup Page Need due to: Authentication Requirements	CO3
10	Designing Signin and Signup Page Need due to: Authentication Requirements	CO3
11	Design Other Product Pages Need due to: Current Need: Improvising Web Page Future need: Routing and nested Routing	CO4
12	Design Other Product Pages Need due to: Current Need: Improvising Web Page Future need: Routing and nested Routing	CO4
13	SPA routing fundamentals; dynamic & nested routing; rendering boundaries; protected routes; Need due to: More Pages and Routing Required Performance Improvement with Increased Page Switching Speed.	CO5
14	SPA routing fundamentals; dynamic & nested routing; rendering boundaries; protected routes; Need due to: More Pages and Routing Required Performance Improvement with Increased Page Switching Speed.	CO5
15	Creating Cart Page Need due to: Project Objective	CO5
16	Creating Cart Page Need due to: Project Objective	CO5
17	form engineering: validation pipelines, controlled forms, error surfaces; Need due to: Improving Project connectivity with end user	CO5
18	accessibility engineering: ARIA, keyboard flow, color contrast; performance tuning. virtualization, memoization, keying, batching renders, lazy loading; architectural tradeoffs in performance engineering. Need due to: Improving Project accessibility	CO5
19	Packaging App (Why bundlers exist; Vite/Webpack engineering flow; bundling, tree-shaking, minification, asset optimization; environment configs; error reporting; linting/formatting;) Need due to: Improving Performance and Easy Deployment	CO6
20	Deployment (CI/CD pipeline design; cloud deployment (Netlify/Vercel); monitoring production; performance evaluation through Lighthouse; engineering practices for maintainability and scalability.) Need due to: Global Accessibility	CO6

Skilling Session wise Teaching – Learning Plan

SESSION NUMBER : 1

Session Outcome: 1 Analyze modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topics	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 2

Session Outcome: 1 Analyze modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 3

Session Outcome: 1 Analyze modern front-end architectures, UI engineering constraints, rendering pipelines, and component-driven engineering principles that form the foundation of React and all contemporary frameworks.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 4

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 5

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 6

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 7

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 8

Session Outcome: 2 Apply JavaScript (ES6+) and TypeScript engineering patterns—including modular design, immutability, functional composition, async pipelines—to construct scalable front-end architectures.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 9

Session Outcome: 3 Build React applications using component-driven development, state and props architecture, side-effect management, reconciliation fundamentals, and controlled UI patterns.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 10

Session Outcome: 3 Build React applications using component-driven development, state and props architecture, side-effect management, reconciliation fundamentals, and controlled UI patterns.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 11

Session Outcome: 4 Apply front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 12

Session Outcome: 4 Apply front-end applications using state management strategies, asynchronous data flows, API integration layers, and architectural patterns

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 13

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topics	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 14

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 15

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	3	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 16

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 17

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 18

Session Outcome: 5 Evaluate routing strategies, form engineering, accessibility (a11y), performance tuning, and rendering optimizations to create highly usable and performant React applications.

Time(min)	Topic	BTL	Teaching-Learning Methods	Active Learning Methods
10	Attendance and Recap	1	Talk	--- NOT APPLICABLE ---
40	Explaining the Topic	2	PPT	--- NOT APPLICABLE ---
50	Demonstrating the Concept	4	LTC	--- NOT APPLICABLE ---

SESSION NUMBER : 19

Session Outcome: 6 Assess front-end applications for production readiness using build systems, bundlers, environment configurations, testing pipelines, CI/CD, and cloud deployment workflows.

	Hour	1	2	3	4	5	6	7	8	9
	Lab	--	--	--	--	--	--	--	--	--
	Skilling	--	--	--	--	--	--	--	--	--
Thu	Theory	--	--	--	--	--	--	--	--	--
	Tutorial	--	--	--	--	--	--	--	--	--
	Lab	--	--	--	--	--	--	--	--	--
	Skilling	--	--	--	--	--	--	--	--	--
Fri	Theory	--	--	--	--	--	--	--	--	--
	Tutorial	--	--	--	--	--	--	--	--	--
	Lab	--	--	--	--	--	--	--	--	--
	Skilling	--	--	--	--	--	--	--	--	--
Sat	Theory	--	--	--	--	--	--	--	--	--
	Tutorial	--	--	--	--	--	--	--	--	--
	Lab	--	--	--	--	--	--	--	--	--
	Skilling	--	--	--	--	--	--	--	--	--
Sun	Theory	--	--	--	--	--	--	--	--	--
	Tutorial	--	--	--	--	--	--	--	--	--
	Lab	--	--	--	--	--	--	--	--	--
	Skilling	--	--	--	--	--	--	--	--	--

REMEDIAL CLASSES:

Supplement course handout, which may perhaps include special lectures and discussions that would be planned, and schedule notified according

SELF-LEARNING:

Assignments to promote self-learning, survey of contents from multiple sources.

S.no	Topics	CO	ALM	References/MOOCs
------	--------	----	-----	------------------

DELIVERY DETAILS OF CONTENT BEYOND SYLLABUS:

Content beyond syllabus covered (if any) should be delivered to all students that would be planned, and schedule notified accordingly.

S.no	Advanced Topics, Additional Reading, Research papers and any	CO	ALM	References/MOOCs
------	--	----	-----	------------------

EVALUATION PLAN:

Evaluation Type	Evaluation Component	Weightage/Marks		Assessment Dates	Duration (Hours)	CO1	CO2	CO3	CO4	CO5	CO6
End Semester Summative Evaluation Total= 40 %	Lab End Semester Exam	Weightage	20		100	3.2	3.2	3.4	3.4	3.4	3.4
		Max Marks	100			16	16	17	17	17	17
	SEM End Project	Weightage	20		100	3.2	3.2	3.4	3.4	3.4	3.4
		Max Marks	100			16	16	17	17	17	17

Evaluation Type	Evaluation Component	Weightage/Marks		Assessment Dates	Duration (Hours)	CO1	CO2	CO3	CO4	CO5	CO6
In Semester Formative Evaluation Total= 20 %	MOOCs Review	Weightage	10		100	1.6	1.6	1.7	1.7	1.7	1.7
		Max Marks	100			16	16	17	17	17	17
	Continuous Evaluation - Lab Exercise	Weightage	10		100	1.6	1.6	1.7	1.7	1.7	1.7
		Max Marks	100			16	16	17	17	17	17
In Semester Summative Evaluation Total= 40 %	Lab In sem-II	Weightage	10		100				3	3	4
		Max Marks	100						30	30	40
	Lab In sem-I	Weightage	10		100	3	3	4			
		Max Marks	100			30	30	40			
	Project-Evaluation	Weightage	20		100	3	3	3	3	4	4
		Max Marks	100			15	15	15	15	20	20

ATTENDANCE POLICY:

Every student is expected to be responsible for regularity of his/her attendance in class rooms and laboratories, to appear in scheduled tests and examinations and fulfill all other tasks assigned to him/her in every course. In every course, student has to maintain a minimum of 85% attendance to be eligible for appearing in Semester end examination of the course, for cases of medical issues and other unavoidable circumstances the students will be condoned if their attendance is between 75% to 85% in every course, subjected to submission of medical certificates, medical case file and other needful documental proof to the concerned departments.

DETENTION POLICY :

In any course, a student has to maintain a minimum of 85% attendance and In-Semester Examinations to be eligible for appearing to the Semester End Examination, failing to fulfill these conditions will deem such student to have been detained in that course.

PLAGIARISM POLICY :

Supplement course handout, which may perhaps include special lectures and discussions

COURSE TEAM MEMBERS, CHAMBER CONSULTATION HOURS AND CHAMBER VENUE DETAILS:

Supplement course handout, which may perhaps include special lectures and discussions

Name of Faculty	Delivery Component of Faculty	Sections of Faculty	Chamber Consultation Day (s)	Chamber Consultation Timings for each day	Chamber Consultation Room No:	Signature of Course faculty:
-----------------	-------------------------------	---------------------	------------------------------	---	-------------------------------	------------------------------

GENERAL INSTRUCTIONS

Students should come prepared for classes and carry the text book(s) or material(s) as prescribed by the Course Faculty to the class.

NOTICES

Most of the notices are available on the LMS platform.

All notices will be communicated through the institution email.

All notices concerning the course will be displayed on the respective Notice Boards.

Signature of COURSE COORDINATOR

(Balajee R M)(6298)(EL&GE)

Signature of Department Prof. Incharge Academics & Vetting Team Member

Department Of CSE-1

HEAD OF DEPARTMENT:

Approval from: DEAN-ACADEMICS

(Sign with Office Seal)

[object HTMLDivElement]